

PO - Safepay RAAST - Payouts

Document Number	OI-PO-20260305
Document status	DRAFT
Document owner	@Mubeen Anwar
Solutions Architect	@Maqsood Ali @Owais Khalid @Saqlain Raza
Project Manager	TBD
Priority	TBD
Region	Pakistan
Operator	RAAST / Safepay
Product	PayOut

SafePay API Doc: [🔗 Get Aggregator - Safepay Raastwire](#)

BRD: [📄 RAAST Payout Integration – Business Requirements Document](#)

1. Overview

Raast is Pakistan's national instant payment network operated by the State Bank of Pakistan (SBP). It enables real-time, interoperable fund transfers between bank accounts and mobile wallets across all participating financial institutions in Pakistan. Safepay Raast exposes an aggregator-grade API layer on top of the Raast network, providing Simpaisa with the infrastructure to initiate payouts and receive settlement notifications through a single set of credentials.

Simpaisa operates as the **Aggregator** in this model. Under the aggregator account, Simpaisa can disburse funds to beneficiaries such as vendors, partners, drivers, or any third party holding a valid Raast IBAN. Payouts use the **same aggregator credentials** (`aggregator_id` and `secret_key`) as pay-ins — no separate onboarding is required for the payout capability.

Payouts are fully asynchronous. The POST creation endpoint returns an initial `P_INITIATED` status and a payout token. The final outcome — settled or failed — arrives via webhook. Polling

the GET payments endpoint is a supported secondary fallback.

Integration Path:

1. **Request aggregator access** — Contact Safepay to receive your `aggregator_id` and `secret_key`.
2. **Store your secret key** — Save `secret_key` in a server-side secret manager. Never expose it client-side.
3. **Verify credentials** — Call `GET /v1/aggregators/{{raast-aggregator-id}}` to confirm your credentials are active before building any payout flows.
4. **Validate beneficiary** — Before initiating any payout, resolve the beneficiary's IBAN and registered account title using the Title Fetch or Account Info utilities (Section 6).
5. **Initiate payout** — Call `POST /v1/aggregators/{{raast-aggregator-id}}/payout` with a unique `request_id`, the payout `amount` (string, in PKR), and the beneficiary `creditor_iban`.
6. **Subscribe to webhooks** — Register a webhook endpoint to receive payout and settlement status events in real time. Without registering to a webhook and events you will not receive respective webhooks.
7. **Reconcile** — Use `trace_reference` and `request_id` to match Safepay webhook events with your internal ledger.

2. Environments & Base URLs

All endpoint paths in this document are relative to the appropriate base URL. Replace `{{baseUrl}}` with the correct environment URL in all cURL examples.

 **Note:** The sandbox environment exists but does not work properly. Safepay has recommended using the **live environment only**.

Environment	Base URL	Notes
Production	<code>https://api.getsafepay.com/raastwire</code>	Live traffic. Real money movement.

3. Authentication

Every API request must include the aggregator secret key as a request header:

```
1 X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}
```

The `secret_key` is the master credential issued by Safepay when your aggregator account is created. It authenticates all operations — payments, payouts, merchant management, and webhooks — under your aggregator identity. This secret key can be changed using Access Key Management (see Section 5).



Security requirement: Store `secret_key` exclusively in a server-side secret manager (e.g., AWS Secrets Manager, HashiCorp Vault). Never embed it in frontend code, mobile apps, or version control. Exposure of this key compromises your entire aggregator account.

4. Get Aggregator (Credential Verification)

This is the first API call you should make after receiving credentials. Call this endpoint to verify that your `aggregator_id` and `secret_key` are valid and that your aggregator account is active before building any payout flows.

Description: Retrieves full metadata for your aggregator account, including active access keys, webhook configuration, and vault status.

Method: GET

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	<code>{{secret_key}}</code>	Aggregator secret key for authentication

raast-aggregator-id	Path	String	M	agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e	Unique aggregator token issued by Safepay
---------------------	------	--------	---	--	---

Sample Request (cURL)

bash

```
1 curl --request GET \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}" \
3   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version
data.id	String	M	"2"	Internal numeric ID of the aggregator
data.token	String	M	"agg_2288490a-..."	Aggregator token; this is your raast-aggregator-id
data.partner_id	String	M	"partner_ed4f0480-..."	Partner identifier
data.bank_account_id	String	M	"acc_c03a57cb-..."	Linked bank account identifier

data.name	String	M	"Safepay"	Display name of the aggregator
data.is_active	Boolean	M	true	Whether the aggregator account is active
data.vault_enabled	Boolean	M	true	Whether vault tokenization is enabled
data.webhook_url	String	O	"www.google.com"	Configured webhook delivery URL
data.webhook_secret	String	O	"0C5FFVV72GRA8Y5AFE6JDE4YFY S4QF"	Secret used to verify incoming webhook signatures
data.Keys[]	Array	M	—	Array of access key objects
data.Keys[].id	String	M	"2"	Internal ID of the access key
data.Keys[].token	String	M	"ak_fa66e6f5-..."	Access key token identifier
data.Keys[].key_id	String	M	"key_452442dc-..."	Unique key ID
data.Keys[].aggregator_id	String	M	"agg_2288490a-..."	Parent aggregator token

data.Keys[].is_active	Boolean	M	true	Whether this key is active
data.Keys[].name	String	M	"Staging key"	Human-readable label for the key
data.Keys[].secret	String	M	"18108aad..."	Key secret value
data.Keys[].created_at	String	M	"2025-03-13T09:12:51Z"	Key creation timestamp (ISO 8601)
data.Keys[].updated_at	String	M	"2025-03-13T09:12:51Z"	Key last-updated timestamp (ISO 8601)
data.created_at	String	M	"2025-03-13T09:12:34Z"	Aggregator account creation timestamp
data.updated_at	String	M	"2025-05-23T09:58:39Z"	Aggregator last-updated timestamp

Sample Response

Success (HTTP 200):

json

```

1 {
2   "api_version": "v1",
3   "data": {
4     "id": "2",
5     "token": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
6     "partner_id": "partner_ed4f0480-36c8-4a5a-8cd4-381bc4d83f2c",
7     "bank_account_id": "acc_c03a57cb-a616-42a7-b1dc-de473217a558",
8     "name": "Safepay",
9     "is_active": true,
10    "vault_enabled": true,
11    "webhook_url": "www.google.com",
12    "webhook_secret": "0C5FFVV726RA8Y5AFE6JDE4YFYS4QF",

```

```
13   "Keys": [  
14     {  
15       "id": "2",  
16       "token": "ak_fa66e6f5-cd88-4d8a-8fe7-0eb1898d20a3",  
17       "key_id": "key_452442dc-d6b3-4cbb-bcc6-8781ef2d3498",  
18       "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",  
19       "is_active": true,  
20       "name": "Staging key",  
21       "secret":  
22         "18108aad30da06075e6f02fab3166926827538363842185faf6e3e694ea5d481",  
23       "created_at": "2025-03-13T09:12:51Z",  
24       "updated_at": "2025-03-13T09:12:51Z"  
25     },  
26     {  
27       "created_at": "2025-03-13T09:12:34Z",  
28       "updated_at": "2025-05-23T09:58:39Z"  
29     }  
  ]  
}
```

Error (HTTP 404):

json

```
1 {  
2   "code": "error.resource_not_found",  
3   "message": "code=404, message=Not Found"  
4 }
```

5. Access Keys Management

Access keys are named credentials scoped to your aggregator account. They support rotation for key hygiene and can be deactivated without revoking the master `secret_key`. Use these APIs to audit active keys, update metadata, or rotate secrets in production.

5a. List Access Keys

Description: Fetches a paginated list of access keys associated with the aggregator. Supports filtering by active status and name. Use this to audit which keys are active and to discover key tokens needed for update or rotate operations.

Method: GET

Endpoint: `/v1/aggregators/{raast-aggregator-id}/keys`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandator y	Sample Value	Descriptio n
-----------	------	-----------	---------------	-----------------	-----------------

X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	{{secret_key}}	Aggregator secret key for authentication
raast-aggregator-id	Path	String	M	agg_2288490a-.. ..	Aggregator token
limit	Query	Integer	O	10	Maximum number of records to return. Default: 10
offset	Query	Integer	O	0	Number of records to skip for pagination. Default: 0
names	Query	String	O	"prodkey"	Filter results by access key name
is_active	Query	Boolean	O	true	Filter by active state (true or false)

Sample Request (cURL)

```

1
2 bash
3 curl --request GET \
4   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/keys?
   limit=10&offset=0&is_active=true" \
5   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
6

```


Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
<code>api_version</code>	String	M	"v1"	API version
<code>data.keys[]</code>	Array	M	—	Array of access key objects
<code>data.keys[].id</code>	String	M	"2"	Internal numeric ID
<code>data.keys[].token</code>	String	M	"ak_fa66e6f5-..."	Access key token; use as <code>{{access-key-id}}</code> in update/rotate calls
<code>data.keys[].key_id</code>	String	M	"key_452442dc-..."	Unique key identifier
<code>data.keys[].aggregator_id</code>	String	M	"agg_2288490a-..."	Parent aggregator token
<code>data.keys[].is_active</code>	Boolean	M	true	Whether this key is currently active
<code>data.keys[].name</code>	String	M	"Staging key"	Human-readable label
<code>data.keys[].secret</code>	String	M	"18108aad..."	Key secret value

<code>data.keys [].create d_at</code>	String	M	<code>"2025-03- 13T09:12: 51Z"</code>	Creation timestamp
<code>data.keys [].update d_at</code>	String	M	<code>"2025-03- 13T09:12: 51Z"</code>	Last-updated timestamp
<code>data.coun t</code>	String	C	<code>"2"</code>	Total count of matching keys (present when filters are applied)

Sample Response

Success (HTTP 200 — Base):

```
1 json
2 {
3   "api_version": "v1",
4   "data": {
5     "keys": [
6       {
7         "id": "2",
8         "token": "ak_fa66e6f5-cd88-4d8a-8fe7-0eb1898d20a3",
9         "key_id": "key_452442dc-d6b3-4cbb-bcc6-8781ef2d3498",
10        "aggregator_id": "agg_2288490a-2176-4de5-b373-
11        0ffb6f8e2e6e",
12        "is_active": true,
13        "name": "Staging key",
14        "secret":
15        "18108aad30da06075e6f02fab3166926827538363842185faf6e3e694ea5d481",
16        "created_at": "2025-03-13T09:12:51Z",
17        "updated_at": "2025-03-13T09:12:51Z"
18      },
19      {
20        "id": "4",
21        "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
22        "key_id": "key_5430e06f-cefb-4393-8b7a-489c8d7c1128",
23        "aggregator_id": "agg_2288490a-2176-4de5-b373-
24        0ffb6f8e2e6e",
25        "is_active": true,
26        "name": "prod key",
27        "secret":
28        "3a7c97a7cb15cb90a46503ab577298e3619ae67e5b885cdd34efc9f24a3816ee",
29        "created_at": "2025-05-23T10:02:26Z",
30        "updated_at": "2025-05-23T10:05:34Z"
31      }
32    ]
33  }
34 }
```

Success (HTTP 200 — With `is_active` filter and count):

```

1
2 json
3 {
4   "api_version": "v1",
5   "data": {
6     "keys": [
7       {
8         "id": "4",
9         "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
10        "key_id": "key_28d69595-4ae4-4645-b9d8-300fa5211845",
11        "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
12        "is_active": true,
13        "name": "prod key",
14        "secret":
"23052bb4602992643b8cd3aaee10a1054a5b071a2ecd3bf2e39ba5072ad3f4cd",
15        "created_at": "2025-05-23T10:02:26Z",
16        "updated_at": "2025-07-02T09:16:55Z"
17      }
18    ],
19    "count": "2"
20  }
21 }
22

```

Success (HTTP 200 — With `names` filter):

```

1
2 json
3 {
4   "api_version": "v1",
5   "data": {
6     "keys": [
7       {
8         "id": "4",
9         "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
10        "key_id": "key_28d69595-4ae4-4645-b9d8-300fa5211845",
11        "aggregator_id": "agg_2288490a-2176-4de5-b373-
0ffb6f8e2e6e",
12        "is_active": true,
13        "name": "prod key",
14        "secret":
"23052bb4602992643b8cd3aaee10a1054a5b071a2ecd3bf2e39ba5072ad3f4cd",
15        "created_at": "2025-05-23T10:02:26Z",
16        "updated_at": "2025-07-02T09:16:55Z"
17      }
18    ],
19    "count": "1"
20  }
21 }
22

```

Error (HTTP 401):

```

1
2 json
3 {
4   "api_version": "v1",
5   "error": {
6     "code": "error.unauthorized_access",

```

```
7      "message": "aggregator not found"
8    }
9  }
10 }
```

5b. Update Access Key

Description: Updates an existing access key's `name` and/or `is_active` status. Use this to label keys meaningfully or to deactivate a compromised key without deleting it. Only `is_active` and `name` are updatable fields.

Method: PUT

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/keys/{{access-key-id}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandator y	Sample Value	Descriptio n
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	{{secret_key}}	Aggregator secret key
raast-aggregator-id	Path	String	M	agg_2288490a-.. ..	Aggregator token
access-key-id	Path	String	M	ak_da8f1c10-.. .	Token of the access key to update
is_active	Body	Boolean	O	true	Set to false to deactivate the key

name	Body	String	O	"prod key"	New display name for the key
------	------	--------	---	---------------	---------------------------------------

Sample Request (cURL)

```

1
2 bash
3 curl --request PUT \
4   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-
5   id}}/keys/{{access-key-id}}" \
6   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
7   --header "Content-Type: application/json" \
8   --data '{
9     "is_active": true,
10    "name": "prod key"
11  }'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version
data.message	String	M	"success"	Confirmation of update

Sample Response

Success (HTTP 200):

```

1
2 json
3 {
4   "api_version": "v1",
5   "data": {
6     "message": "success"
7   }
8 }
9
```

5c. Rotate Access Key

Description: Rotates an existing access key by generating a new secret while invalidating the old one. The key token (`ak_...`) remains the same; only the `secret` value changes. Use this as part of regular key hygiene or after a suspected secret exposure.

Method: PUT

Endpoint: `/v1/aggregators/{raast-aggregator-id}/keys/{access-key-id}/rotate`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {secret_key}`

Request Parameters

Parameter	Type	Data Type	Mandator y	Sample Value	Descriptio n
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	{{secret_key}}	Aggregator secret key
raast-aggregator-id	Path	String	M	agg_2288490a-.. ..	Aggregator token
access-key-id	Path	String	M	ak_da8f1c10-.. .	Token of the access key to rotate

Sample Request (cURL)

```
1
2 bash
3 curl --request PUT \
4   --url "{{baseUrl}}/v1/aggregators/{raast-aggregator-
5   id}/keys/{access-key-id}/rotate" \
6   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {secret_key}"
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version
data.id	String	M	"4"	Internal key ID
data.token	String	M	"ak_da8f1c10-..."	Access key token (unchanged after rotation)
data.key_id	String	M	"key_5430e06f-..."	Unique key identifier
data.aggregator_id	String	M	"agg_2288490a-..."	Parent aggregator
data.is_active	Boolean	M	true	Key active status
data.name	String	M	"prod key"	Key label
data.secret	String	M	"3a7c97a7..."	New rotated secret value — store this immediately
data.created_at	String	M	"2025-05-23T10:02:26Z"	Original creation timestamp
data.updated_at	String	M	"2025-05-23T10:05:15Z"	Rotation timestamp

Sample Response

Success (HTTP 200):

```
1
2 json
3 {
4   "api_version": "v1",
5   "data": {
6     "id": "4",
7     "token": "ak_da8f1c10-346a-4be8-895a-b6187a9a9662",
8     "key_id": "key_5430e06f-cefb-4393-8b7a-489c8d7c1128",
9     "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
10    "is_active": true,
11    "name": "prod key",
12    "secret":
13    "3a7c97a7cb15cb90a46503ab577298e3619ae67e5b885cdd34efc9f24a3816ee",
14    "created_at": "2025-05-23T10:02:26Z",
15    "updated_at": "2025-05-23T10:05:15Z"
16  }
17 }
```

Important: The new `secret` value is returned only once in the rotation response. Store it in your secret manager immediately. Subsequent calls to List Access Keys will still show the key, but the plain-text secret will not be re-exposed.

6. Utilities

Use these utility endpoints to validate debtor or beneficiary account details **before** initiating any RTP payment or payout. Validating account details upfront reduces failed transactions and avoids unnecessary charges.

6a. Title Fetch (by IBAN)

Description: Returns the officially registered account title for a Raast IBAN by querying the Central Account Service (CAS). Use this before initiating an RTP or payout to confirm the destination account holder name matches the expected beneficiary. This API will only title-fetch for the IBAN to confirm the validity of the account entered.

Method: GET

Endpoint: /v1/aggregators/{raast-aggregator-id}/title-fetch?iban={{iban}}

Authentication: Header — X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-SECRET-KEY	Header	String	M	{{secret_key}}	Aggregator secret key
raast-aggregator-id	Path	String	M	agg_2288490a-... ..	Aggregator token
iban	Query	String	M	PK89SCBL0000001234651601	Full Pakistani IBAN to resolve

Sample Request (cURL)

```
1 |
2 | bash
3 | curl --request GET \
4 |   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/title-
   fetch?iban=PK89SCBL0000001234651601" \
5 |   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
6 |
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version

data.account_title	String	M	"Muhammad Ali"	Registered account holder name from CAS
data.account_type	String	M	"Account"	Type of bank account
data.iban	String	M	"PK89SCBL..."	Resolved IBAN

Sample Response

Success (HTTP 200):

```

1  json
2  {
3    "api_version": "v1",
4    "data": {
5      "iban": "PK89SCBL0000001234651601",
6      "account_title": "Muhammad Ali",
7      "account_type": "Account"
8    }
9  }
10 }
11

```

6b. Account Info (by IBAN)

Description: Given an IBAN, this endpoint retrieves account information and — when `with_title_fetch=true` — additionally queries CAS to return the registered account title and account type. Use this when the debtor or beneficiary is identified by a direct IBAN.

Method: GET

Endpoint: `/v1/aggregators/{raast-aggregator-id}/account-info?iban={{iban}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandator y	Sample Value	Descriptio n
------	-----------	-----------	---------------	-----------------	-----------------

Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	{{secret_key}}	Aggregator secret key
Path	raast-aggregator-id	String	M	agg_2288490a-.. ..	Aggregator token
Query	iban	String	M	PK91ALFH0005001007079203	Target account IBAN
Query	with_title_fetch	Boolean	O	true	If true, also fetches account title and type from CAS. Default: false

Sample Request (cURL)

```

1
2 bash
3 curl --location --globoff 'https://api.getsafepay.com/raastwire{{raast-wire}}/v1/aggregators/{{raast-aggregator-id}}/account-info?iban=PK91ALFH0005001007079203&with_title_fetch=true' \
4   --header 'X-SFPY-AGGREGATOR-SECRET-KEY: {{X-SFPY-AGGREGATOR-SECRET-KEY}}'
5

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version

data.iban	String	M	"PK91ALFH 000500100 7079203"	Resolved IBAN
data.type	String	M	"CAS_CONS T_UNSPECI FIED"	Alias type descriptor
data.currenc y	String	M	"PKR"	Account currency
data.servicer _member_id	String	M	"ALFHPKKA "	Servicer member identifier
data.name	String	M	"MIRZA"	Account holder name
data.is_defau lt	Boolean	M	false	Whether this is the default alias
data.dba	String	O	""	Doing- business-as name
data.mcc	String	O	""	Merchant category code
data.account _title	String	C	"Title-1 100707920 3"	Account holder title (present when with_titl e_fetch=t rue)
data.account _type	String	C	"Account"	Account type (present when with_titl

				e_fetch=true)
--	--	--	--	----------------

Sample Response

Success (HTTP 200):

```

1  json
2  {
3    {
4      "api_version": "v1",
5      "data": {
6        "iban": "PK91ALFH0005001007079203",
7        "type": "CAS_CONST_UNSPECIFIED",
8        "currency": "PKR",
9        "servicer_member_id": "ALFHPKKA",
10       "name": "MIRZA",
11       "is_default": false,
12       "dba": "",
13       "mcc": "",
14       "account_title": "Title-1 1007079203",
15       "account_type": "Account"
16     }
17   }
18 }

```

6c. Account Info (by Alias — Raast ID)

Description: Given a Raast alias (Mobile number), this endpoint resolves the associated IBAN and — when `with_title_fetch=true` — additionally queries Safepay to return the registered account title and account type. Use this when the debtor or beneficiary is identified by a mobile number alias rather than a direct IBAN. Through this Title Fetch of a Raast ID will also be done.

Method: GET

Endpoint: `/v1/aggregators/{raast-aggregator-id}/account-info?alias_type={{alias_type}}&alias_value={{alias_value}}`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Parameter	Type	Data Type	Mandatory	Sample Value	Description
X-SFPY-AGGREGATOR-	Header	String	M	{{secret_key}}	Aggregator secret key

SECRET-KEY					
raast-aggregator-id	Path	String	M	agg_2288490a-.. ..	Aggregator token
alias_type	Query	String	M	MOBILE	Alias type (e.g. MOBILE)
alias_value	Query	String	M	"03472840476"	Mobile number linked to the Raast alias
with_title_fetch	Query	Boolean	O	true	If true, also fetches account title and type from CAS. Default: false

Sample Request (cURL)

```

1
2 bash
3 curl --location --globoff 'https://api.getsafepay.com/raastwire{{raast-wire}}/v1/aggregators/{{raast-aggregator-id}}/account-info?with_title_fetch=true&alias_type=MOBILE&alias_value=03472840476' \
4   --header 'X-SFPY-AGGREGATOR-SECRET-KEY: {{X-SFPY-AGGREGATOR-SECRET-KEY}}'
5

```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version

data.iban	String	M	"PK74S0NE 000022000 8098833"	Resolved IBAN for the alias
data.type	String	M	"CAS_CONS T_UNSPECI FIED"	Alias type descriptor
data.currenc y	String	M	"PKR"	Account currency
data.servicer _member_id	String	M	"SONEPKKA "	Servicer member identifier
data.name	String	M	"ZAIN BIN OMER"	Account holder name linked to this alias
data.is_defau lt	Boolean	M	true	Whether this is the default alias
data.dba	String	O	" "	Doing- business-as name
data.mcc	String	O	" "	Merchant category code
data.account _title	String	C	"ZAIN BIN OMER"	Account holder title (present when with_titl e_fetch=t rue)
data.account _type	String	C	"Account"	Account type (present

				when with_titl e_fetch=t rue)
--	--	--	--	---

Sample Response

Success (HTTP 200):

```

1  json
2  {
3    "api_version": "v1",
4    "data": {
5      "iban": "PK74SONE0000220008098833",
6      "type": "CAS_CONST_UNSPECIFIED",
7      "currency": "PKR",
8      "servicer_member_id": "SONEPKKA",
9      "name": "ZAIN BIN OMER",
10     "is_default": true,
11     "dba": "",
12     "mcc": "",
13     "account_title": "ZAIN BIN OMER",
14     "account_type": "Account"
15   }
16 }
17

```

7. Payouts

Payouts allow Simpaisa to disburse funds to beneficiaries (vendors, partners, drivers, commission recipients, etc.) using the same aggregator credentials as pay-ins. All payouts are **asynchronous**; always validate the beneficiary IBAN using Title Fetch (Section 6) before submitting a payout.

Payout Flow Summary:

1. **Validate beneficiary** — Run `GET /title-fetch` or `GET /account-info` to confirm the IBAN resolves to the expected account holder.
2. **Initiate payout** — Call `POST /v1/aggregators/{raast-aggregator-id}/payout` with a unique `request_id`, the payout `amount` (string, in PKR), and the beneficiary `creditor_iban`.
3. **Await webhook** — Safepay pushes status updates to your registered webhook endpoint. Polling is optional but recommended as a safety net.
4. **Reconcile and notify** — Update internal ledgers using `trace_reference` and `request_id`, and archive the payload for audit.

 **Amount format note:** The amount parameter is set to paisas instead of rupees. So “200” means PKR 2 rather than PKR 200

7a. Create Payout

Description: Initiates a funds transfer to a beneficiary IBAN over the Raast network. Use a unique request_id per payout for idempotency. Returns initial status P_INITIATED; final outcome is delivered via webhook.

Method: POST

Endpoint: /v1/aggregators/{{raast-aggregator-id}}/payout

Auth Header: X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}

Request Parameters

Type	Parameter	Data Type	M/O/C	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	{{secret_key}}	Aggregator secret key for authentication
Path	raast-aggregator-id	String	M	"agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e"	The unique identifier for the aggregator
Body	request_id	String	M	"3fa85f64-5717-..."	Unique identifier supplied by the client to

					ensure idempotency. Reuse only when retrying the same payout
Body	amount	String	M	"200"	Amount to credit in PKR, expressed as a string to preserve formatting. "200" = PKR 200. Note: This is PKR not paisa — different from Pay-ins where amount is in paisa
Body	creditor_iban	String	M	"PK89SCB L0000001 23465160 1"	Destination IBAN that receives the payout. Must be validated via Title Fetch (Section 6)

					before submissio n
--	--	--	--	--	--------------------------

Sample Request (cURL)

```

1 curl --request POST \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payout" \
3   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
4   --header "Content-Type: application/json" \
5   --data '{
6     "request_id": "{{guid}}",
7     "amount": "200",
8     "creditor_iban": "PK89SCBL0000001234651601"
9   }'
```

Response Parameters

Parameter	Data Type	Sample Value	Description
api_version	String	"v1"	The current version of the API spec
data.ID	String	"44"	Internal identifier for the payment record
data.token	String	"pm_2a8c2cfc-..."	Public token that uniquely identifies the payment. Use this as payment_token when polling status (Section 7b)
data.aggregator_id	String	"agg_2288490a-..."	Aggregator identifier that owns the payment

data.aggregator_merchant_id	String	""	Token from merchant creation
data.qr_code_id	String	""	Identifier of the QR code associated with the payment. Empty for payouts
data.order_id	String	""	Merchant-supplied order identifier. Empty for payouts
data.type	Enum (String)	"PAYOUT"	Indicates how the payment was initiated. Always "PAYOUT" for payout transactions. Available values: RTP_NOW, RTP_LATER, STATIC_QR, DYNAMIC_QR, PAYOUT
data.amount	String	"200"	Payment amount expressed as a string in PKR. "200" = PKR 200
data.status	Enum (String)	"P_INITIATED"	Current status of the payment. See Section 7c for full status reference and

			recommended actions. Available values: P_INITIATED, P_RECEIVED, P_CANCELLED, P_REJECTED, P_FAILED, P_AUTHORIZED, P_REVERSED, P_CAPTURED, P_SETTLED, P_PARTIALLY_REFUNDED, P_REFUNDED
data.uetr	String	"3335131b-..."	Unique End-to-end Transaction Reference assigned by Raast. Store this for support escalations
data.trace_reference	String	"2005867c-..."	Trace reference identifier used for reconciliation. Store this to match webhook events to internal ledger entries
data.msg_id	String	"QAI8456FZ5..."	Raast network message identifier for the payment
data.instr_id	String	"cb865267..."	Instruction identifier from

			the Raast message payload
data.end_to_end_id	String	"3335131b..."	End-to-end identifier recorded for the payment
data.pmt_inf_id	String	""	Payment information identifier from the Raast message
data.request_id	String	"d44940a1-..."	Client-supplied idempotency identifier echoed back from your request
data.msg_created_at	String	"2025-10-02 11:03:11.041"	Timestamp captured in the Raast message associated with the payment
data.expires_at	String (datetime)	"1970-01-01T00:00:00Z"	Expiry timestamp for the payment request. Not applicable for payouts
data.created_at	String (datetime)	"2025-10-02T06:03:11Z"	Timestamp when the payment record was created (ISO 8601 UTC)

data.updated_at	String (datetime)	"2025-10-02T06:03:11Z"	Timestamp when the payment record was last updated (ISO 8601 UTC)
data.txn_logs[]	Array	—	Lifecycle events generated for the payment
data.txn_logs[].id	String	"35"	Identifier of the transaction log entry
data.txn_logs[].type	String	"TLT_PAYMENT"	Classification of the logged event
data.txn_logs[].payment_id	String	"pm_2a8c2cfc-..."	Identifier of the payment linked to this log event
data.txn_logs[].status	String	"P_INITIATED"	Payment status recorded by this log entry
data.txn_logs[].refund_id	String	""	Refund identifier when the log entry references a refund
data.txn_logs[].settlement_id	String	""	Settlement identifier when the log entry references settlement activity
data.txn_logs[].txn_desc	String	""	Human-readable description of the event

data.txn_logs[].additional_info	String	""	Additional information provided for the event
data.txn_logs[].token	String	"txnlog_b9679d70-..."	Public token assigned to the log entry
data.txn_logs[].timestamp	String (datetime)	"2025-10-02T06:03:11Z"	ISO 8601 timestamp captured for the event
data.txn_logs[].iso_msg_type	String	"pain.013.001.09"	Raast ISO message type associated with this event
data.payment_parties[]	Array	—	Participant records (debtor and creditor) involved in the payment
data.payment_parties[].id	String	"87"	Internal Safepay identifier for the payment party
data.payment_parties[].token	String	"pp_6a71861c-..."	Public token representing the payment party
data.payment_parties[].role	String	"ROLE_DEBITOR" or "ROLE_CREDITOR"	Role played by the participant. ROLE_DEBITOR = Simpaisa's account (source of funds). ROLE_CREDITOR = beneficiary

			account (destination)
data.payment_parties[].payment_id	String	"pm_2a8c2cfc-..."	Identifier of the payment this party belongs to
data.payment_parties[].bank_account_id	String	"acc_c03a57cb-..."	Identifier of the linked bank account
data.payment_parties[].bank_account.id	String	"11"	Internal Safepay identifier for the bank account
data.payment_parties[].bank_account.token	String	"acc_c03a57cb-..."	Public token for the bank account
data.payment_parties[].bank_account.vault_token	String	"lock_464309f2-..."	Token referencing the encrypted/vaulted bank account
data.payment_parties[].bank_account.title	String	"SYED MUHAMMAD HAQANI"	Registered account holder name
data.payment_parties[].bank_account.iban	String	""	Full IBAN. May be empty; use redacted_iban for any display purposes
data.payment_parties[].bank_account.redacted_iban	String	"PK84HABB*** *****8594"	Masked IBAN safe for display and logging
data.payment_parties[].bank_account.hash	String	"TAQswoYY..."	Hashed IBAN for secure comparison

data.payment_parties[].bank_account.servicer	String	"HABBPKKARTG"	BIC/servicer code of the bank
data.payment_parties[].bank_account.bank	String	""	Bank name. May be empty
data.payment_parties[].bank_account.depositor_id	String	""	Depositor identifier if applicable
data.payment_parties[].bank_account.owner	String	""	Owner reference if applicable
data.payment_parties[].bank_account.kind	Integer	—	Account kind classification
data.payment_parties[].bank_account.version	String	—	Account record version
data.payment_parties[].bank_account.created_at	String (datetime)	"2025-03-13T09:12:35Z"	Bank account creation timestamp (ISO 8601 UTC)
data.payment_parties[].bank_account.updated_at	String (datetime)	"2025-03-13T09:12:35Z"	Bank account last updated timestamp (ISO 8601 UTC)
data.payment_parties[].created_at	String (datetime)	"2025-10-02T06:03:11Z"	Payment party record creation timestamp (ISO 8601 UTC)
data.payment_parties[].updated_at	String (datetime)	"2025-10-02T06:03:11Z"	Payment party record last updated

			timestamp (ISO 8601 UTC)
data.debitor	Object	null	Debitor metadata when available
data.qr_code	Object	null	QR code details. Always null for payouts
data.aggregator_merchant	Object	null	Aggregator merchant details when linked to the payment. Null when payout is initiated at aggregator level directly
data.refunds[]	Array	[]	Refunds applied to the payment. Empty array on initial payout creation
data.refunds[].token	String	—	Public token for the refund
data.refunds[].payment_id	String	—	Identifier of the payment being refunded
data.refunds[].amount	String	—	Refund amount as a string
data.refunds[].type	String	—	Refund type classification
data.refunds[].reason	String	—	Reason provided for the refund

data.refunds[].additional_info	String	—	Additional information about the refund
data.refunds[].status	String	—	Current status of the refund
data.refunds[].trace_reference	String	—	Trace reference for reconciliation of the refund
data.refunds[].message_id	String	—	Raast network message ID for the refund
data.refunds[].return_id	String	—	Return identifier assigned to the refund
data.refunds[].request_id	String	—	Client-supplied idempotency ID for the refund request
data.refunds[].message_created_at	String (datetime)	—	Timestamp of the Raast message for the refund (ISO 8601 UTC)
data.refunds[].settle_at	—	—	Settlement timestamp for the refund
data.refunds[].created_at	String (datetime)	—	Refund record creation timestamp (ISO 8601 UTC)
data.refunds[].updated_at	String (datetime)	—	Refund record last updated

			timestamp (ISO 8601 UTC)
data.settlement	Object	null	Settlement information for the payment when applicable. Null at payout creation; populated after P_SETTLED

Sample Response — Success (HTTP 200)

```

1  {
2    "api_version": "v1",
3    "data": {
4      "id": "44",
5      "token": "pm_2a8c2cfc-e61d-431e-a850-f1808405ca07",
6      "aggregator_id": "agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e",
7      "aggregator_merchant_id": "",
8      "qr_code_id": "",
9      "order_id": "",
10     "type": "PAYOUT",
11     "amount": "200",
12     "status": "P_INITIATED",
13     "uetr": "3335131b-b26c-41c6-a089-a3b4905dcc68",
14     "trace_reference": "2005867c-752f-430f-8619-a4c998423e3d",
15     "msg_id": "QAI8456FZ5NOR5H6ITEVK2U2NCD7POCSKQS",
16     "instr_id": "cb865267aec142d389f3ff728bb2492a",
17     "end_to_end_id": "3335131bb26c41c6a089a3b4905dcc68",
18     "pmt_inf_id": "",
19     "request_id": "d44940a1-663b-4d92-91e7-65438e4bf80b",
20     "msg_created_at": "2025-10-02 11:03:11.041",
21     "expires_at": "1970-01-01T00:00:00Z",
22     "created_at": "2025-10-02T06:03:11Z",
23     "updated_at": "2025-10-02T06:03:11Z",
24     "txn_logs": [
25       {
26         "id": "35",
27         "type": "TLT_PAYMENT",
28         "payment_id": "pm_2a8c2cfc-e61d-431e-a850-f1808405ca07",
29         "status": "P_INITIATED",
30         "refund_id": "",
31         "settlement_id": "",
32         "txn_desc": "",
33         "addtl_info": "",
34         "token": "txnlog_b9679d70-8874-459b-ad55-28180c53f0b1",
35         "timestamp": "2025-10-02T06:03:11Z",
36         "iso_msg_type": "pain.013.001.09"
37       }
38     ],
39     "payment_parties": [
40       {
41         "id": "87",

```

```

42     "token": "pp_6a71861c-2e6a-4355-957e-0b6d8e4d9a06",
43     "role": "ROLE_DEBITOR",
44     "payment_id": "pm_2a8c2cfc-e61d-431e-a850-f1808405ca07",
45     "bank_account_id": "acc_c03a57cb-a616-42a7-b1dc-de473217a558",
46     "bank_account": {
47         "id": "11",
48         "token": "acc_c03a57cb-a616-42a7-b1dc-de473217a558",
49         "vault_token": "lock_464309f2-572f-493f-8272-e503277b06fd",
50         "title": "SYED MUHAMMAD HAQANI",
51         "iban": "",
52         "redacted_iban": "PK84HABB*****8594",
53         "hashed_iban":
"TAQswoYYIJaiPH15TAS5+tnFe7yDOK0kWP2wgSXwi7k=",
54         "servicer": "HABBPkkARTG",
55         "bank": "",
56         "created_at": "2025-03-13T09:12:35Z",
57         "updated_at": "2025-03-13T09:12:35Z"
58     },
59     "created_at": "2025-10-02T06:03:11Z",
60     "updated_at": "2025-10-02T06:03:11Z"
61 },
62 {
63     "id": "88",
64     "token": "pp_42dd9241-794b-4fad-b4c2-ca8d88ce8ca6",
65     "role": "ROLE_CREDITOR",
66     "payment_id": "pm_2a8c2cfc-e61d-431e-a850-f1808405ca07",
67     "bank_account_id": "acc_9fd8d208-0e51-4869-87ea-12bbfc3a3ad6",
68     "bank_account": {
69         "id": "81",
70         "token": "acc_9fd8d208-0e51-4869-87ea-12bbfc3a3ad6",
71         "vault_token": "lock_124456f6-3e1d-4f31-8baa-ea914ed365f0",
72         "title": "Title-1 1008658216",
73         "iban": "",
74         "redacted_iban": "PK25ALFH*****8216",
75         "hashed_iban":
"7zWv4dnBcoTKS4/dyrbMSWlFKVuwNWVkv1KmeRiu7Us=",
76         "servicer": "ALFHPKKA",
77         "bank": "",
78         "created_at": "2025-09-22T09:42:41Z",
79         "updated_at": "2025-09-22T09:42:41Z"
80     },
81     "created_at": "2025-10-02T06:03:11Z",
82     "updated_at": "2025-10-02T06:03:11Z"
83 }
84 ],
85 "debitor": null,
86 "qr_code": null,
87 "aggregator_merchant": null,
88 "refunds": [],
89 "settlement": null
90 }
91 }

```

Sample Response — Error (HTTP 401 — Unauthorized)

```

1  {
2    "api_version": "v1",
3    "error": {
4      "code": "error.unauthorized_access",
5      "message": "partner access key is required in request header"
6    }
7  }

```

Sample Response — Error (HTTP 500 — Internal Server Error)

```

1 {
2   "api_version": "v1",
3   "error": {
4     "code": "error.internal_server_error",
5     "message": "rpc error code Internal desc = something went wrong"
6   }
7 }

```

7b. Get Payout Status(One Specific Payment) (Poll)

Description: Retrieves the current status and full detail of a previously initiated payout. Use this endpoint for fallback polling when a webhook has not been received, or to populate dashboard views.

Method: GET

Endpoint: /v1/aggregators/{raast-aggregator-id}/payments/{payment_token}

Authentication: Header — X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}

Request Parameters

Type	Parameter	Data Type	Mandator y	Sample Value	Descriptio n
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	{{secret_key}}	Aggregator secret key
Path	raast-aggregator-id	String	M	agg_2288490a-.. ..	Aggregator token
Path	payment_token	String	M	pm_2a8c2cfc-.. .	Payment token from the create payout response

Sample Request (cURL)

bash

```

1 curl --request GET \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-
3   id}}/payments/{{payment_token}}" \
   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Sample Response

Success (HTTP 200):

```

1 json
2 {
3   {
4     "api_version": "v1",
5     "data": {
6       "id": "641",
7       "token": "pm_b4a30cc5-f0ec-4c61-8fe3-3ece42196c81",
8       "aggregator_merchant_id": "am_d7fd05ea-8613-4038-afd7-
9       4275f29110f5",
10      "qr_code_id": "",
11      "order_id": "NewPayLater1",
12      "type": "RTP_NOW",
13      "amount": "34000",
14      "status": "P_CAPTURED",
15      "uetr": "f4bbe7db-9f5f-4fe3-92e1-2de04f492713",
16      "trace_reference": "0c843220-2813-4573-83e6-180ad99c2933",
17      "msg_id": "ZCPC3GS3LJNPMXQBN7G3LXU7WDJJ6PB2JVK",
18      "instr_id": "fabce92286f14ab3852aed0e3e5d28ec",
19      "end_to_end_id": "f4bbe7db9f5f4fe392e12de04f492713",
20      "pmt_inf_id": "AG9G53RMNQRLFKX1AZCT6WG3GD9SZ6080Q7",
21      "request_id": "d4d5d335-af3e-4de7-9724-26fad5026366",
22      "msg_created_at": "2025-06-26 15:58:25.",
23      "expires_at": "2025-07-03T10:58:26Z",
24      "created_at": "2025-06-26T10:58:26Z",
25      "updated_at": "2025-06-27T09:49:48Z",
26      "txn_logs": [],
27      "debitor": null,
28      "qr_code": null,
29      "aggregator_merchant": null,
30      "refunds": [],
31      "settlement": null
32    }
33  }
34
35
36

```

7b. List Payments(ALL) (Poll Payout Status)

Description: Returns a paginated list of payments under the aggregator account. For Payouts, use this as a fallback polling mechanism when a webhook has not been received, or to query the status of multiple payouts at once. Filter by type PAYOUT to retrieve all payout transactions. Use request_id, trace_reference, or uetr to locate a specific payout transaction.

Method: GET

Endpoint: /v1/aggregators/{{raast-aggregator-id}}/payments

Auth Header: X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}

Request Parameters

Type	Parameter	Data Type	M/O/C	Sample Value	Description
Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	{{secret_key}}	Aggregator secret key for authentication
Path	raast-aggregator-id	String	M	"agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e"	The unique identifier for the aggregator
Query	limit	String	O	"20"	Maximum number of payments to return in the result set
Query	offset	String	O	"0"	Number of payments to skip before starting the result set. Use with limit for pagination

Query	aggregator_merchant_id	String	O	"am_749fe6ce-..."	Filter by Safepay aggregator merchant identifier. Not applicable for Payouts as there is no sub-merchant management in the payout flow
Query	order_id	String	O	"order03330"	Filter by the merchant supplied order identifier
Query	request_id	String	O	"6a049190-..."	Filter by the client generated idempotent request identifier. Use this to locate a specific payout
Query	trace_reference	String	O	"ce08b498-..."	Filter by the trace reference assigned

					during processing. Use this to locate a specific payout
Query	msg_id	String	O	"FE0PMU5HH..."	Filter by the Raast network message identifier
Query	pmt_inf_id	String	O	"9545TGYRG0..."	Filter by the payment information identifier from the Raast message
Query	uetr	String	O	"e3391812-..."	Filter by the Unique End-to-end Transaction Reference (UETR). Use this to locate a specific payout
Query	status	Enum (String)	O	"P_INITIATED"	Filter by payment status. Available

					values: P_INITIATED, P_RECEIVED, P_CANCELLED, P_REJECTED, P_FAILED, P_AUTHORIZED, P_REVERSED, P_CAPTURED, P_SETTLED, P_PARTIALLY_REFUNDED, P_REFUNDED
Query	type	Enum (String)	O	"PAYOUT"	Filter by payment type. Use PAYOUT to retrieve all payout transactions. Available values: RTP_NOW, RTP_LATER, STATIC_Q

					R, DYNAMIC _QR, PAYOUT
Query	payment_id	String	O	"644"	Filter by the internal payment identifier
Query	qr_code_id	String	O	—	Filter by the QR code identifier associated with the payment. Not applicable for Payouts
Query	end_to_end_id	String	O	—	Filter by the end-to-end identifier recorded for the payment
Query	instr_id	String	O	—	Filter by the instruction identifier from the Raast message payload

Query	start_date	Integer	O	1719446400	Filter payments created on or after this date (UNIX timestamp)
Query	end_date	Integer	O	1719532800	Filter payments created on or before this date (UNIX timestamp)

Sample Request (cURL)

```

1 curl --request GET \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/payments?
   limit=20&offset=0&type=PAYOUT" \
3   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
api_version	String	M	"v1"	The current version of the API spec
data.count	String	M	"605"	Total number of payments matching the supplied filters

data.payments[]	Array	M	—	List of payment objects matching the supplied filters
data.payments[].id	String	M	"644"	Internal identifier for the payment record
data.payments[].token	String	M	"pm_f37a10dd-..."	Public token that uniquely identifies the payment. Use this as payment_token for individual payment lookups
data.payments[].aggregator_id	String	M	"agg_2288490a-..."	Aggregator identifier that owns the payment
data.payments[].aggregator_merchant_id	String	O	""	Aggregator merchant on whose behalf the payment was initiated. Always empty for Payouts — there is no sub-merchant

				management in the payout flow
data.payments[].qr_code_id	String	O	""	Identifier of the QR code associated with the payment. Empty for Payouts
data.payments[].order_id	String	O	"has_3320"	Merchant-supplied order identifier for the payment
data.payments[].type	Enum (String)	M	"PAYOUT"	Indicates how the payment was initiated. For payout transactions this will always be PAYOUT. Available values: RTP_NOW, RTP_LATER, STATIC_QR, DYNAMIC_QR, PAYOUT
data.payments[].amount	String	M	"250"	Payment amount expressed as a string in

				paisa. "250" = PKR 2.50
data.payments[].status	Enum (String)	M	"P_INITIATED"	Current status of the payment. See Section 7c for full status reference and recommended actions. Available values: P_INITIATED, P_RECEIVED, , P_CANCELLED, P_REJECTED, , P_FAILED, P_AUTHORIZED, P_REVERSED, P_CAPTURED, P_SETTLED, P_PARTIALLY_REFUNDED, P_REFUNDED
data.payments[].uetr	String	M	"e3391812-..."	Unique End-to-end

				Transaction Reference assigned by Raast
data.payments[].trace_reference	String	M	"ce08b498-.."	Trace reference identifier used for reconciliation
data.payments[].msg_id	String	M	"FE0PMU5HH..."	Raast network message identifier for the payment
data.payments[].instr_id	String	O	—	Instruction identifier from the Raast message payload
data.payments[].end_to_end_id	String	O	—	End-to-end identifier recorded for the payment
data.payments[].pmt_inf_id	String	O	"9545TGYRGO..."	Payment information identifier from the Raast message
data.payments[].request_id	String	M	"6a049190-.."	Client-supplied idempotency identifier

data.payments[msg_created_at]	String	O	—	Timestamp captured in the Raast message associated with the payment
data.payments[expires_at]	String (datetime)	O	"2025-06-27T06:47:34Z"	Expiry timestamp for the payment request
data.payments[created_at]	String (datetime)	M	"2025-06-27T06:45:35Z"	Timestamp when the payment record was created (ISO 8601 UTC)
data.payments[updated_at]	String (datetime)	M	"2025-06-27T06:48:33Z"	Timestamp when the payment record was last updated (ISO 8601 UTC)
data.payments[txn_logs[]]	Array	O	—	Lifecycle events generated for the payment. See Section 7a for full txn_logs child

				attribute definitions
data.payments[].payment_parties[]	Array	O	—	Participant records involved in the payment lifecycle. See Section 7a for full payment_parties child attribute definitions
data.payments[].debitor	Object	O	—	Debitor metadata when available
data.payments[].qr_code	Object	O	null	QR code details when available. Null for Payouts
data.payments[].aggregator_merchant	Object	O	null	Aggregator merchant details when linked to the payment. Null for Payouts
data.payments[].refunds[]	Array	O	[]	Refunds applied to the payment
data.payments[].settlement	Object	O	null	Settlement information

nt				when applicable. Populated after P_SETTLED
----	--	--	--	--

Sample Response — Success (HTTP 200)

```
1 {
2   "api_version": "v1",
3   "data": {
4     "payments": [
5       {
6         "id": "644",
7         "token": "pm_f37a10dd-c530-4d86-8483-be473e820578",
8         "aggregator_merchant_id": "",
9         "order_id": "",
10        "type": "PAYOUT",
11        "amount": "250",
12        "status": "P_INITIATED",
13        "uetr": "e3391812-f9f5-4feb-bd6d-60698e9b662f",
14        "trace_reference": "ce08b498-376c-41dd-b32b-f814441c4507",
15        "msg_id": "FE0PMU5HHKKPF4S7PLR20M8HKBYI57WAUV5",
16        "pmt_inf_id": "9545TGyRG0F6DCQEF2ZH7UC0CU57FUNYDL9",
17        "request_id": "6a049190-3595-47cb-8603-7e310c3adb15",
18        "expires_at": "2025-06-27T06:47:34Z",
19        "created_at": "2025-06-27T06:45:35Z",
20        "updated_at": "2025-06-27T06:48:33Z"
21      },
22      {
23        "id": "643",
24        "token": "pm_49707018-026f-4159-8b1a-da0e3b482cbb",
25        "aggregator_merchant_id": "",
26        "order_id": "",
27        "type": "PAYOUT",
28        "amount": "22000",
29        "status": "P_SETTLED",
30        "uetr": "8fe838bb-57f7-4488-8198-b7b240eac83c",
31        "trace_reference": "5041299c-c5bd-4ac0-b3f8-b6131f15c55f",
32        "msg_id": "54QMCJLI7UATIC1MZYXPXESGMJ505TXUSX",
33        "pmt_inf_id": "LWMEZ7RUGR92H0TJKMUSK0CC381NK9Z2K8C",
34        "request_id": "8f20f196-e1c9-46b7-9b41-67f17baa7438",
35        "expires_at": "1970-01-01T00:00:00Z",
36        "created_at": "2025-06-26T11:21:23Z",
37        "updated_at": "2025-06-26T11:21:23Z"
38      }
39    ],
40    "count": "605"
41  }
42 }
```

7c. Payout Status Reference

Status	Description	Recommended Action
--------	-------------	--------------------

P_INITIATED	Safepay accepted the payout and queued it for processing	Display "Processing" in dashboards
P_RECEIVED	Beneficiary bank is validating the transfer	No action required; continue waiting
P_SETTLED	Funds have reached the beneficiary	Mark payout complete and notify recipient
P_FAILED	Payout could not be delivered (invalid account, compliance block, insufficient balance)	Investigate failure reason, fix data, and retry with same <code>request_id</code> if appropriate

7d. Webhook Events to Expect — Payouts

Event	Description
<code>payment.created</code>	Payout request was accepted and initiated
<code>payment.completed</code>	Payout has been processed successfully
<code>payment.settled</code>	Funds have settled to the beneficiary
<code>payment.failed</code>	Payout processing failed
<code>payment.reversed</code>	Payout was reversed after completion
<code>settlement.created</code>	Settlement batch was created
<code>settlement.processing</code>	Settlement batch is currently processing
<code>settlement.completed</code>	Settlement completed successfully
<code>settlement.failed</code>	Settlement failed during processing

<code>settlement.on_hold</code>	Settlement temporarily placed on hold
<code>settlement.reversed</code>	Settlement was reversed

7e. Failure Handling Checklist

- **Inspect webhook payloads** for failure reasons (insufficient funds, invalid account, compliance hold).
- **Retry transient errors** using the same `request_id` and identical payload to stay idempotent.
- **Raise manual review tasks** for permanent failures (`P_FAILED`) and notify beneficiaries.
- **Do not retry with a new** `request_id` unless the original payout definitively failed and you have confirmed no funds moved. Sending a new `request_id` for the same intended transfer can result in a duplicate disbursement.

7f. Troubleshooting — Payouts

Issue	Resolution
Invalid beneficiary IBAN	Always run Title Fetch (Section 6) before submitting a payout.
<code>P_FAILED</code> — insufficient balance	Check aggregator ledger balance. Ensure your account has adequate funds before initiating payouts.
Duplicate payout	Reusing a <code>request_id</code> with a different body will trigger an idempotency error. Retry with the exact same payload .
Payout stuck in <code>P_RECEIVED</code> beyond SLA	Trigger an alert. Contact Safepay support with the <code>trace_reference</code> and <code>uetr</code> .
Amount validation error	Confirm <code>amount</code> is passed as a string in PKR (e.g., <code>"500"</code>), not an integer and not in paisa.

7g. Operational Best Practices

- **Pre-validate beneficiaries** — Always run Title Fetch before submitting a payout to avoid `P_FAILED` outcomes from invalid IBANs.
- **Webhook reliability** — Implement exponential backoff on your webhook receiver if your endpoint experiences downtime.
- **Dual controls** — Require two-person review for high-value payouts before calling the API.
- **Ledger reconciliation** — Store both `trace_reference` and `request_id` to match Safepay webhook events to your internal ledger entries.
- **Incident alerting** — Configure automatic alerts if a payout remains in `P_RECEIVED` beyond your SLA, or if it enters `P_FAILED`.
- **Idempotent retries** — Retry transient failures using the same `request_id` and identical payload to stay idempotent.
- **Settlement windows** — Raast payments settle in batches every 6 hours. The `running_balance` in ledger accounts reflects activity within the current batch and resets to `0` after settlement completes.

9. Webhooks

Webhooks are the **primary source of truth** for all asynchronous payout outcomes. Simpaisa must maintain a publicly accessible HTTPS endpoint to receive Safepay event notifications.

9a. Create Webhook

Description: Registers a new webhook endpoint to receive event notifications. Specify the list of event types to subscribe to. The response includes a `webhook_secret` used to verify incoming payloads.

Method: `POST`

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/webhooks`

Authentication: Header — `X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}`

Request Parameters

Type	Parameter	Data Type	Mandatory	Sample Value	Description

Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	<code>{{secret_key}}</code>	Aggregator secret key
Body	url	String	M	<code>"https://api.simpaisa.com/webhooks/safepay"</code>	HTTPS endpoint to receive event payloads
Body	description	String	O	<code>"Raast payout webhook endpoint"</code>	Human-readable description
Body	event_types	Array	M	<code>["payment.created", "payment.completed", "payment.settled"]</code>	List of event types to subscribe to
Body	enabled	Boolean	M	<code>true</code>	Whether this webhook is active

Sample Request (cURL)

bash

```
1 curl --request POST \
```

```

2  --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/webhooks"
  \
3  --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}" \
4  --header "Content-Type: application/json" \
5  --data '{
6    "url": "https://api.simpaisa.com/webhooks/safepay",
7    "description": "Raast payout webhook endpoint",
8    "event_types": [
9      "payment.created",
10     "payment.completed",
11     "payment.settled",
12     "payment.failed",
13     "payment.reversed",
14     "settlement.created",
15     "settlement.processing",
16     "settlement.completed",
17     "settlement.failed"
18   ],
19   "enabled": true
20 }'
```

Response Parameters

Parameter	Data Type	Mandatory	Sample Value	Description
api_version	String	M	"v1"	API version
data.token	String	M	"wh_XXXXX XXX-..."	Webhook endpoint token
data.url	String	M	"https:// api.simpa isa.com/. ..."	Registered endpoint URL
data.event_types	Array	M	["payment .complete d"]	Subscribed event types
data.enabled	Boolean	M	true	Active status
data.webhook_secret	String	M	"whsec_ab c123..."	Secret for HMAC signature verification — store immediately

data.created_at	String	M	"2025-07-29T05:00:00Z"	Creation timestamp
-----------------	--------	---	------------------------	--------------------

9b. List Webhooks

Description: Returns all webhook endpoints registered for the aggregator.

Method: GET

Endpoint: /v1/aggregators/{{raast-aggregator-id}}/webhooks

Authentication: Header — X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}

Sample Request (cURL)

bash

```
1 curl --request GET \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-id}}/webhooks" \
3   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"
```

9c. Webhook Headers

Every event delivery from Safepay includes the following HTTP headers:

Header	Description
X-SFPY-SIGNATURE	HMAC-SHA256 signature of the payload for authenticity verification
X-SFPY-TIMESTAMP	ISO 8601 timestamp of when the event was dispatched
X-SFPY-EVENT-ID	Unique identifier for the event delivery
X-SFPY-EVENT-TYPE	Event type string (e.g., payment.completed)
X-SFPY-AGGREGATOR-ID	Your aggregator ID, confirming the intended recipient

8d. Signature Verification

Safepay signs each webhook payload using HMAC-SHA256. **Always verify signatures before processing any event** to prevent spoofed requests.

Verification algorithm:

1. Extract `X-SFPY-SIGNATURE` and `X-SFPY-TIMESTAMP` from the incoming request headers.
2. Construct the signing string: `timestamp + "." + raw_payload_bytes`.
3. Compute HMAC-SHA256 using your `webhook_secret` as the key.
4. Compare the computed digest to the provided `X-SFPY-SIGNATURE` using a **constant-time comparison** to prevent timing attacks.
5. Optionally enforce a timestamp tolerance (e.g., ± 5 minutes) to reject replayed events.

Go reference implementation:

go

```
1 func Verify(secret, body []byte, providedSig, providedTS string,
2 tolerance time.Duration) error {
3     if tolerance > 0 {
4         parsed, err := time.Parse(time.RFC3339Nano, providedTS)
5         if err != nil {
6             return err
7         }
8         if delta := time.Since(parsed); delta > tolerance || delta < -
9 tolerance {
10             return errTimestampDrift
11         }
12     }
13     mac := hmac.New(sha256.New, secret)
14     mac.Write([]byte(providedTS))
15     mac.Write([]byte{'.'})
16     mac.Write(body)
17     expected := fmt.Sprintf(headerFormat,
18 hex.EncodeToString(mac.Sum(nil)))
19     if !hmac.Equal([]byte(expected), []byte(providedSig)) {
20         return errSignatureMismatch
21     }
22     return nil
23 }
```

 **Note:** Use the raw (unparsed) request body as the `body` argument. Parsing and re-serializing JSON can alter byte ordering and invalidate the signature.

8e. Retry Behavior

If your endpoint does not return an HTTP `200 OK` response, Safepay will retry the delivery using exponential backoff:

Attempt	Delay
1	1 second
2	2 seconds
3	4 seconds
4	8 seconds
5	16 seconds

After 5 failed attempts, the delivery is **abandoned**. Implement **idempotent processing** in your webhook handler — the same event may arrive more than once. Return `200 OK` immediately upon persisting the event to storage; process asynchronously to avoid timeouts.

8f. Full Event Catalog — Payout-Relevant Events

Event	Category	Description
<code>payment.created</code>	Payments	A new payout request was accepted
<code>payment.completed</code>	Payments	Payout has been captured or processed successfully
<code>payment.settled</code>	Payments	Payout funds have settled to the beneficiary
<code>payment.failed</code>	Payments	Payout processing failed
<code>payment.reversed</code>	Payments	Payout was reversed after completion
<code>settlement.created</code>	Settlements	Settlement request was created

<code>settlement.processing</code>	Settlements	Settlement is currently processing
<code>settlement.completed</code>	Settlements	Settlement completed successfully
<code>settlement.failed</code>	Settlements	Settlement failed during processing
<code>settlement.on_hold</code>	Settlements	Settlement temporarily placed on hold
<code>settlement.reversed</code>	Settlements	Settlement was reversed

Full event catalog (including payment and refund events available for subscription):

Event	Category	Description
<code>payment.pending_authorization</code>	Payments	Payment is awaiting authorization
<code>payment.authorized</code>	Payments	Payment has been authorized and funds are on hold
<code>payment.refunded</code>	Payments	Payment was fully refunded
<code>payment.refund_partial</code>	Payments	Payment was partially refunded
<code>payment.rejected</code>	Payments	Payment was rejected before authorization
<code>payment.voided</code>	Payments	Payment authorization was voided
<code>refund.created</code>	Refunds	Refund request was created

<code>refund.completed</code>	Refunds	Refund was successfully completed
<code>refund.failed</code>	Refunds	Refund failed during processing
<code>refund.canceled</code>	Refunds	Refund request was canceled

9. Ledger

The Ledger API provides a financial audit trail of all transactions processed under the aggregator account, including payouts and settlements. Use this for reconciliation, financial reporting, and compliance audits.

i Ledger accounts are not bank accounts themselves — they are a real-time representation of what Safepay owes or is owed, backed by settlement runs to your bank account. Raast payments settle in batches every **6 hours**. The `running_balance` reflects activity within the current batch and resets to `0` after settlement completes.

9a Find Aggregator Ledger Account

Description: Retrieves the full details of Simpaisha's aggregator ledger account. Use this to inspect the current settled balance and running balance before initiating payouts. This is the aggregator-level account only — there are no merchant-level ledger accounts in the payout flow.

Method: GET

Endpoint: `/v1/aggregators/{{raast-aggregator-id}}/ledger/accounts`

Auth Header: X-SFPY-AGGREGATOR-SECRET-KEY: `{{secret_key}}`

Request Parameters

Type	Parameter	Data Type	M/O/C	Sample Value	Description
------	-----------	-----------	-------	--------------	-------------

Header	X-SFPY-AGGREGATOR-SECRET-KEY	String	M	{{secret_key}}	Aggregator secret key for authentication
Path	raast-aggregator-id	String	M	"agg_2288490a-2176-4de5-b373-0ffb6f8e2e6e"	The unique identifier for the aggregator

Sample Request (cURL)

```

1 curl --request GET \
2   --url "{{baseUrl}}/v1/aggregators/{{raast-aggregator-
3   id}}/ledger/accounts" \
   --header "X-SFPY-AGGREGATOR-SECRET-KEY: {{secret_key}}"

```

Response Parameters

Parameter	Data Type	M/O/C	Sample Value	Description
api_version	String	M	"v1"	The current version of the API spec
data.id	String	M	"3"	Internal numeric identifier for the ledger account
data.token	String	M	"acct_a4a2e caa-..."	Public unique token for the ledger

				account. Use this as a stable reference in reconciliation and reporting
data.account_type	String	M	"ACCT_AGGREGATOR"	Type of ledger account. For Payouts this will always be ACCT_AGGREGATOR — confirming funds are held and disbursed at the aggregator level
data.account_name	String	M	"Simpaisa"	Human-readable display name for the ledger account
data.normal_balance	String	M	"SIDE_CREDIT"	Natural balance side for the account. SIDE_CREDIT means the account is naturally in credit. Used internally by

				Safepay for applying double-entry ledger entries
data.balance	String	M	"0"	Current settled balance in PKR. Check this before initiating payouts to ensure sufficient funds are available
data.currency	String	M	"PKR"	ISO 4217 currency code
data.running_balance	String	M	"0"	Balance including pending activity within the current 6-hour settlement batch. Resets to 0 after each settlement cycle completes
data.vault_to_ken	String	M	"lock_81f9c5db-..."	-

data.bank_account_id	String	M	"acc_ff3555cc-..."	Identifier of the physical bank account in Raast Wire that is linked to this ledger account
data.external_partner_id	String	M	"partner_ed4f0480-..."	Reference to Simpaisa's upstream partner record in Raast Wire
data.external_aggregator_id	String	M	"agg_10951914-..."	Simpaisa's aggregator identifier used to link this ledger account to the integration
data.external_merchant_id	String	O	""	Merchant identifier when the ledger account is merchant-scoped. Always empty for aggregator-level payout accounts

data.created_at	String (datetime)	M	"2025-10-26T21:39:39Z"	Timestamp when the ledger account was created (ISO 8601 UTC)
data.updated_at	String (datetime)	M	"2025-12-16T18:39:17Z"	Timestamp of the most recent ledger update (ISO 8601 UTC)

Sample Response — Success (HTTP 200)

```

1 {
2   "api_version": "v1",
3   "data": {
4     "id": "3",
5     "token": "acct_a4a2ecaa-5d14-4330-970b-89c0d5c30934",
6     "account_type": "ACCT_AGGREGATOR",
7     "account_name": "Simpaisa",
8     "normal_balance": "SIDE_CREDIT",
9     "balance": "0",
10    "currency": "PKR",
11    "running_balance": "0",
12    "vault_token": "lock_81f9c5db-3e7b-47f1-80b6-a1de8a762de8",
13    "bank_acct_id": "acc_ff3555cc-149a-4883-9945-401170d0480d",
14    "external_partner_id": "partner_ed4f0480-36c8-4a5a-8cd4-
15    381bc4d83f2c",
16    "external_aggregator_id": "agg_10951914-7e84-40e9-9693-
17    19a22c8ede89",
18    "external_merchant_id": "",
19    "created_at": "2025-10-26T21:39:39Z",
20    "updated_at": "2025-12-16T18:39:17Z"
21  }
22 }
```

Sample Response — Error (HTTP 401 — Unauthorized)

```

1 {
2   "api_version": "v1",
3   "error": {
4     "code": "error.unauthorized_access",
5     "message": "partner access key is required in request header"
6   }
7 }
```
